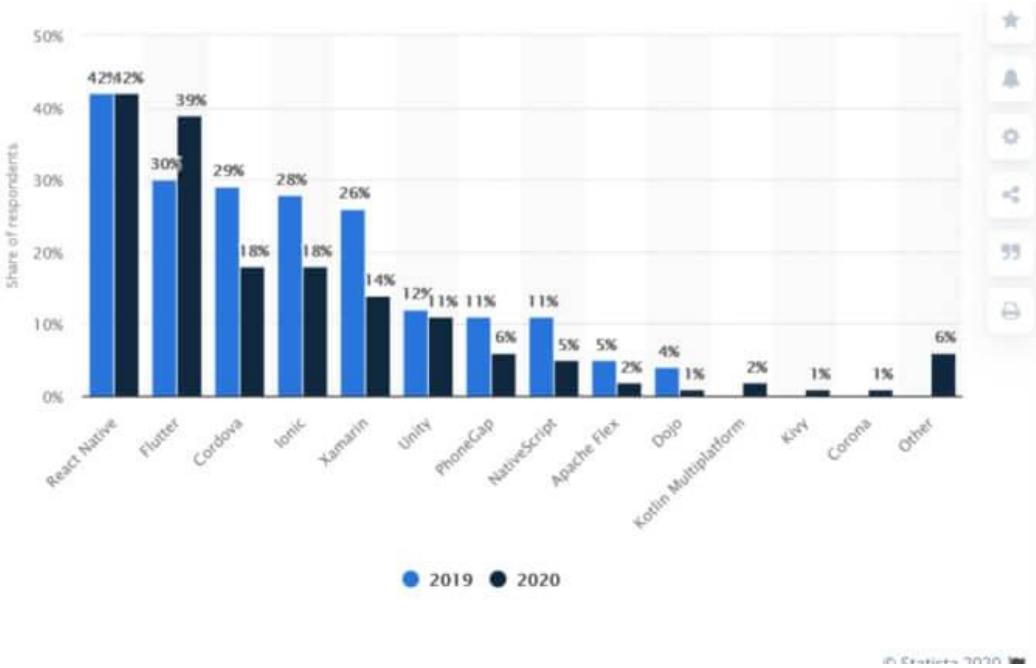
พื้นฐานภาษา



https://www.insightssuccess.com/top-5-mobile-app-development-frameworks-2020-2021/

Flutter มีพื้นฐานของการใช้ Dart ในการพัฒนา ดังนั้นควรศึกษาการใช้งานตั้งแต่ตัวภาษา Dart แล้วค่อยต่อยอดสำหรับการพัฒนาแอปพลิเคชัน ดังนั้นบทเรียนนี้จะเป็นการแนะนำภาษา Dart เบื้องต้น

1. Variables and Operators
2. Expressions
3. Decision Making and Looping Constructs
4. Comments
5. Functions
6. Classes
7. Libraries and Packages
8. Typedefs
9. Data structures represented as Collections / Generics

Variables and Operators

เริ่มต้นด้วยฟังก์ชัน main()

```
main() {
    <statements>
}
```

ฟังก์ชันนี้เป็นส่วนแสดงผลของโปรแกรม ในภาษา Dart ใช้ main() ในการกำหนดตัวแปรหรือใช้ฟังก์ชันใด ๆ ก็ตาม
ตัวแปร (Variable) หมายถึง การตั้งชื่อ space ในเมโมรี่ของเครื่อง แล้วบรรจุหรือกำหนดค่า (value)
ชื่อของตัวแปรมีศัพท์เทคนิค(จะถูกเรียก)ว่า “identifier”
กฎการตั้งชื่อตัวแปร

- ห้ามเป็น keyword
- ประกอบด้วย พยัญชนะ หรือ สระ ห้ามมีสัญลักษณ์พิเศษ ได้แก่ underscore (_) และ dollar sign ($)
- ห้ามขึ้นต้นด้วยตัวเลข
- เป็น case-sensitive ยกตัวอย่าง john จะไม่เท่ากับ John
- ห้ามมีช่องว่าง เช่น “John Doe”

Variable

การประกาศตัวแปร จะมีการใช้ keyword ก่อนเสมอ แล้วตามด้วยชื่อตัวแปร
Keyword เป็นได้ทั้ง Access Modifier (var, final, const) และ Data Type

Data Type

การประกาศตัวแปรโดยใช้ Data Type ดังนี้

DATA TYPE	KEYWORD	DESCRIPTION
Numbers	int, double	int : เลขฐาน 10 จำนวนเต็ม / double เลขฐาน 10 ติดทศนิยม
Strings	String	ข้อความ หรือ อักษร
Booleans	bool	true หรือ false
Lists	List	สร้างรายการ Array
Maps	Map	สร้าง JSON Object
Dynamic	dynamic	สร้างอะไรก็ได้

สามารถประกาศตัวแปรแบบใส่ type ก่อนหรือไม่ใส่ก็ได้
ถ้าระบุ type : ใช้ Data Type แล้วตามด้วยชื่อตัวแปร
data_type variable_name

หรือจะใช้ Access Modifier (final, const) ร่วมด้วยก็ได้
access_modifier data_type variable_name

ถ้าไม่ระบุ type : ใช้ Access Modifier (var, final, const) แล้วตามด้วยชื่อตัวแปรเลย
โดย type จะเป็นไปตามชนิดของ value ที่กำหนด
access_modifier variable_name

```
var hero_1 = 'Abaddon'          //String
var id = 123                     //Number
final num_list = [1,2,3];       //Lists
```

ข้อควรระวัง ถ้าประกาศ var ซ้อนกันกับ Data Type จะ **error!**

Note 1 : สังเกตว่า ตัว Access Modifier (var) กับ Data Type (dynamic) สามารถใช้กำหนดตัวแปรชนิดไหนก็ได้
แต่ทั้งสองมีข้อแตกต่างกันตรงที่ dynamic เปลี่ยน type ได้เรื่อย ๆ เช่น

```
dynamic a = 1;                  // เป็น int
a = true;                       // ตอนนี้เป็น boolean
```

ส่วน var กำหนดตัวแปรเป็น type อะไรแล้ว ถ้าจะเปลี่ยนค่า ก็ต้องเป็น type เดิม เช่น

```
var b = 2;                      // เป็น int
b = 3;                          // ยังเป็น int อยู่
b = 'handsome';                 // error เพราะเปลี่ยน type จาก int เป็น String ไม่ได้
```

Note 2 : ตัวแปรที่เป็น const ต่างจาก final ตรงที่ เฉพาะ const เท่านั้นจะถูกคำนวณ "compile time constant"

compile time constant เป็นค่าคงที่ ที่ถูกกำหนดตอนคอมไพล์ (compile time)
ใน Dart มีแค่ const ตัวเดียว ที่เหลือจะถูกสร้างตอนรัน (run time) ทั้งหมด

Operator

คือตัวดำเนินการทางคณิตศาสตร์ที่ใช้ประเมินผล (evaluate) หรือกำหนดค่า (assign) แบ่งออกเป็นกลุ่มตามการใช้งานได้ดังนี้

Arithmetic Operators

Operator ใน Dart ก็เหมือนกับภาษาอื่น ๆ คือมี + (บวก), - (ลบ), * (คูณ), / (หาร type double), ~/ (หาร type integer), % (modulo: หาเศษจากหาร), ++ (increment), -- (decrement)

นอกจากนั้น ก็มี -() (Unary minus : เปลี่ยนค่าเป็นค่าตรงข้าม เช่น -(2) => 2)

Equality and Relational Operators

เอาไว้กำหนดความสัมพันธ์ระหว่างตัวแปรสองตัว เช็คค่าตัวแปร ทำให้ได้คำตอบออกมาเป็น Boolean (true, false)

ได้แก่ > (มากกว่า), < (น้อยกว่า), => (เท่ากับหรือมากกว่า), =< (เท่ากับหรือน้อยกว่า), == (เท่ากับ), != (ไม่เท่ากับ)

Type test Operators

เอาไว้เช็ค type ของตัวแปร ตอน runtime ได้แก่ is และ is!

```
var e;
print(e is int);                    // false
print(e is! int);                   // true
```

แต่สำหรับ String ต้องใส่ค่าก่อน (ไม่เป็น null) ไม่งั้นเช็ค type จะเป็น false

```
String a;
print(a is String);         // false
String b = 'aaa';
print(b is String);         // true
```

Bitwise Operators

คือ ตัวดำเนินการแบบบิต เป็นตัวดำเนินการที่ใช้กับชนิดข้อมูลเลขจำนวนเต็ม (integer) เท่านั้น

โดยจะแปลงเป็นเลขฐาน 2 (8-bit) ก่อนดำเนินการ หลังจากนั้น จะแปลงค่ากลับเป็นฐาน 10 กลับมา ซึ่งมีตัวเนินการได้แก่

& (AND) เป็นการหาเลข 1 ที่ตำแหน่งบิตเดียวกัน มีเงื่อนไขดังนี้ ถ้า 1&1 เป็น 1, ถ้า 1&0 เป็น 0, ถ้า 0&1 เป็น 0, ถ้า 0&0 เป็น 0

ยกตัวอย่าง

```
5 & 3  มีค่าเท่ากับ 1  เนื่องจาก
5 แปลงเป็นเลขฐานสองได้เป็น          0000 0101
3 แปลงเป็นเลขฐานสองได้เป็น          0000 0011
ผลลัพธ์เมื่อ AND กันแต่ละบิต ได้     0000 0001          มีค่าเท่ากับ 1 (เลขฐานสิบ)
```

| (Or) เป็นการตัวเลข 1 อย่างน้อย 1 ตามตำแหน่งเข้าด้วยกัน มีเงื่อนไขดังนี้
ถ้า 1|1 เป็น 1, ถ้า 1|0 เป็น 1, ถ้า 0|1 เป็น 1, ถ้า 0|0 เป็น 0
ยกตัวอย่าง

```
5 | 3  มีค่าเท่ากับ 7  เนื่องจาก
5 แปลงเป็นเลขฐานสองได้เป็น          0000 0101
3 แปลงเป็นเลขฐานสองได้เป็น          0000 0011
ผลลัพธ์เมื่อ OR กันแต่ละบิต ได้      0000 0111        มีค่าเท่ากับ 7 (เลขฐานสิบ)
```

^ (XOR) เป็นการตัวเลข 0 อย่างน้อย 1 ตามตำแหน่งเข้าด้วยกัน มีเงื่อนไขดังนี้
ถ้า 1^1 เป็น 0, ถ้า 1^0 เป็น 1, ถ้า 0^1 เป็น 1, ถ้า 0^0 เป็น 1
ยกตัวอย่าง

```
5 | 3  มีค่าเท่ากับ 6  เนื่องจาก
5 แปลงเป็นเลขฐานสองได้เป็น          0000 0101
3 แปลงเป็นเลขฐานสองได้เป็น          0000 0011
ผลลัพธ์เมื่อ XOR กันแต่ละบิต ได้     0000 0110        มีค่าเท่ากับ 6 (เลขฐานสิบ)
```

~ (NOT) เป็นการกลับค่าในแต่ละตำแหน่ง จาก 1 เป็น 0 และจาก 0 เป็น 1

```
~3  มีค่าเท่ากับ -4  เนื่องจาก
3 แปลงเป็นเลขฐานสองได้เป็น          0000 0011
3 จะถูกกลับค่าได้เป็น                1111 1100        บิตแรก ด้านซ้ายสุด  เป็น 1  จะมีค่า เป็น ติดลบ –
ทำ complement นำ บิต ชุดแรก มาบวก 1  0000 0011 + 1
จะได้ผลลัพธ์เป็น                     0000 0100        เติมค่าติดลบ จาก กลับบิตซ้ายสุดเป็น 1
สรุป ผลลัพธ์ ของ ~3  มีค่าเท่ากับ -4 (เลขฐานสิบ)
```

<< (Left shift) หมายถึงการเลื่อนบิตไปทางซ้ายมือ เช่น

```
5<<2 มีค่าเท่ากับ 20 เนื่องจาก
5 แปลงเป็นเลขฐานสองได้เป็น                    0000 0101
เลื่อนบิตซ้ายสุดของตัวตั้งไปเป็นจำนวน 2 บิต ได้เป็น   00 | 0001 0100
ผลลัพธ์ของการเลื่อนบิตทางขวาของ 5<<2 คือ      0001 0100 มีค่าเท่ากับ 20
(คือการตัดเลขทางซ้ายสุด แล้วเอา 0 มาต่อทางขวาสุดให้ครบ 8 bit)
```

>> (Right shift) หมายถึงการเลื่อนบิตไปทางขวามือ เช่น

```
5>>2 มีค่าเท่ากับ 1 เนื่องจาก
5 แปลงเป็นเลขฐานสองได้เป็น                    0000 0101
เลื่อนบิตขวาสุดของตัวตั้งไปเป็นจำนวน 2 บิต ได้เป็น   0000 0001 | 01
ผลลัพธ์ของการเลื่อนบิตทางขวาของ 5>>2 คือ      0000 0001 มีค่าเท่ากับ 1
(คือการตัดเลขทางขวาสุด แล้วเอา 0 มาต่อทางซ้ายสุดให้ครบ 8 bit)
```

Assignment Operators

เป็นตัวดำเนินการที่เป็นการกำหนดค่าให้ตัวแปร มีดังนี้

= (Simple Assignment)
คือการกำหนดค่าตัวแปรให้เป็นค่าต่าง ๆ คืนค่าเป็นค่าที่กำหนดให้

??=
กำหนดค่าให้ตัวแปรถ้าตัวแปรนั้นเป็น null ถ้าตัวแปรนั้นไม่ใช่ null ค่าของตัวแปรนั้นจะคงเดิม

+= (Add and Assignment)
คือการบวกค่าตัวแปรนั้นด้วยค่าที่กำหนด คืนค่าเป็นตัวแปรที่ผ่านการบวกแล้ว

-= (Subtract and Assignment)
คือการบวกค่าตัวแปรนั้นด้วยค่าที่กำหนด คืนค่าเป็นตัวแปรที่ผ่านการลบแล้ว

***= (Multiply and Assignment)**
คือการบวกค่าตัวแปรนั้นด้วยค่าที่กำหนด คืนค่าเป็นตัวแปรที่ผ่านการคูณแล้ว

/= (Divide and Assignment)
คือการบวกค่าตัวแปรนั้นด้วยค่าที่กำหนด คืนค่าเป็นตัวแปรที่ผ่านการหารแล้ว

Logical Operators

ใช้รวมเงื่อนไขสองเงื่อนไขเข้าด้วยกัน จะคืนค่าออกมาเป็น boolean ตัวดำเนินการมีดังนี้
&& (And) จะเป็น true ถ้า ทั้งสองเงื่อนไขเป็นจริง
|| (Or) จะเป็น true ถ้า เงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง
! (Not) กลับค่า boolean เช่น จาก true เป็น false

Expressions and Programming Constructs

Expression คือ statement ที่ประเมินผลที่มาจาก Operands และ Operator ได้ผลเป็น ค่า (value) ออกมา
ถ้าพิมพ์ expression บน command line interpreter จะทำการประเมินผลและแสดงผลลัพธ์ เช่น

```
>>> 1 + 2
3
```

ในตัวอย่าง Operands คือ 1 กับ 2 ส่วน Operator คือ +

แม้ว่า expression จะประกอบไปด้วย Operands และ Operator แต่ไม่จำเป็นที่ทุก expression ต้องประกอบไปด้วย elements ครบทั้งสอง แค่ Operands อย่างเดียว จะถูกพิจารณาว่าเป็น expression ได้เช่นเดียวกัน เช่น

```
>>> 17
17
>>> x
2
```

และการประเมินผลของ expression นั้นจะไม่เหมือนกับการสั่ง print
ตัวอย่าง

```
>>> message = " What’s up, Doc?"
>>> message
" What’s up, Doc?"
```

Conditional Expressions

Syntax ของการใส่เงื่อนไขเพื่อแสดง expression นอกจาก การใช้ if....else แบบทั่วไปแล้ว ยังสามารถเขียนในรูปแบบต่าง ๆ ดังต่อไปนี้ได้อีก เช่น *condition ? expr1 : expr2*

ตัวอย่าง

```
var a = 50;
var b = a>30 ? a+12 : a;
print(b);                    // output = 62
```

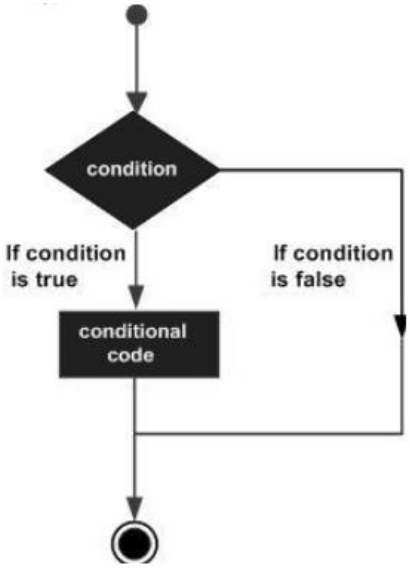
expr1 ?? expr2

ตรวจสอบว่า ถ้า expr1 ไม่ใช่ null จะคืนค่านั้นออกมา ถ้าเป็น null จะคืนค่าจาก expr2
ตัวอย่าง

```
var a;
var b = a ?? 800;
print(b);                    // output = 800
```

syntax ที่ใช้สำหรับสร้างทางเลือกเพื่อตัดสินใจ และการใช้คำสั่งวนซ้ำ

Decision Making



https://www.tutorialspoint.com/dart_programming/dart_programming_decision_making.htm

การสร้างทางเลือกในการตัดสินใจ โดยจะพิจารณาเงื่อนไขก่อนที่จะไปรันคำสั่งนั้น
ในภาษา Dart เองก็มีการสร้างเงื่อนไขเหมือนกับภาษาอื่น ๆ เช่น การใช้ *if* แบบปกติ

```
if(boolean_expression){
   // statement(s) will execute if the Boolean expression is true.
}
```

การใช้ *if...else* เพื่อสร้างทางเลือก 2 ทาง ถ้าทางเลือกแรกเป็นเท็จ ถึงจะรัน statement ทางเลือกที่สอง

```
if(boolean_expression){
   // statement(s) will execute if the Boolean expression is true.
} else {
   // statement(s) will execute if the Boolean expression is false.
}
```

การใช้ *if....if else.....else* จะคล้ายกันกับ *if...else* แต่เป็นการสร้างทางเลือกมากกว่า 2 ทาง และจะดูเงื่อนไขเรียงลำดับไปเรื่อย ๆ จนถึงอันที่ถูก จากนั้นก็จะรันคำสั่ง แล้วออกจาก decision

```
if (boolean_expression1) {
   //statements if the expression1 evaluates to true
}
else if (boolean_expression2) {
   //statements if the expression2 evaluates to true
}
else {
   //statements if both expression1 and expression2 result to false
}
```

Switch Case เป็น statement ที่ใช้สร้างทางเลือกมากกว่า 2 ทาง อีกแบบหนึ่ง คือ

- ใช้ switch เป็น statement หลัก ตามด้วยตัวแปรที่เก็บค่า expression เพื่อเข้าสู่การพิจารณาเงื่อนไข
- ใช้ case เป็นทางเลือกต่าง ๆ และพิจารณา case ที่จะ match กับ expression กับ switch ตามลำดับ จากบนลงล่าง
- ทางเลือกสุดท้าย จะใช้เป็น default แทน โดย ไม่ต้องมี expression เหมือนกับ case
- ถ้า case ใดเป็นจริง จะรันคำสั่งที่กำหนดใน case นั้น ๆ และออกจาก statement ด้วย break
- ถ้าไม่ใช้ break กำกับต่อท้ายของแต่ละ case โปรแกรมจะไม่หยุดทำงาน และดู case ต่อไปอีก

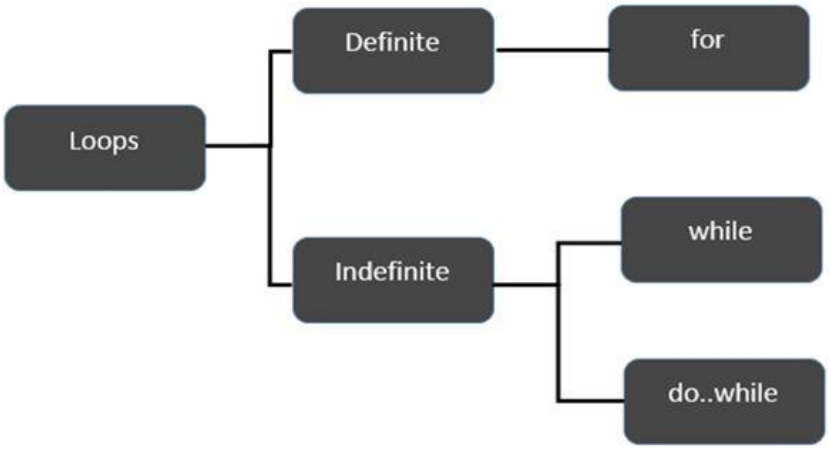
```
switch(variable_expression) {
  case constant_expr1: {
    // statements;
  }
  break;
  case constant_expr2: {
    //statements;
  }
  break;
  default: {
    //statements;
  }
  break;
}
```

ตัวอย่าง Switch Case

```
print('What do you do, guys?');
String bob = 'Rob a bank.';
print(bob);

switch(bob) {
 case 'Study English.' : {print('May you have a good future.');}
 break;
 case 'Do nothing.' : {print('It\'s up to you.');}
 break;
 case 'Rob a bank.' : {print('Please go to jail, Police!!!');}
 break;
 default : {print('Are you a OK?');}
 break;
}
/*
 Output
 What do you do, guys?
 Rob a bank.
 Please go to jail, Police!!!
*/
```

คำสั่งวนซ้ำ (Loop)



https://www.tutorialspoint.com/dart_programming/dart_programming_loops.htm

เราสามารถจำแนกรูปแบบคำสั่งวนซ้ำ ได้ 2 ประเภทใหญ่ ๆ คือ

1. แบบ Definite เป็นการกำหนดจำนวณรอบวนซ้ำ ซึ่งใช้กับรูปแบบ for
2. แบบ Indefinite ไม่กำหนดจำนวณรอบ ซึ่งใช้กับรูปแบบ while และ do...while

รูปแบบ for

สามารถใช้ syntax ได้ 2 แบบ คือ แบบ semicolon (;) คั่น และ แบบที่ใช้ in (เหมือนภาษา Python)

1. แบบ semicolon

```
for (initial_count_value; termination-condition; step) {
  //statements
}
```

initial_count_value คือ expression ระบุค่าตั้งต้น บอกให้โปรแกรมรู้ว่าจะเริ่มที่ค่าอะไร
termination-condition คือ expression ที่เป็นจุดสิ้นสุด คำสั่งวนซ้ำจะทำงานไปจนกว่าจะเป็นเท็จ
step คือ expression ที่บอกว่าคำสั่งวนซ้ำทำงานจบลง 1 ครั้งนั้น จำนวณรอบมีการเปลี่ยนแปลงยังไง

ตัวอย่าง

```
var num = 5;
var factorial = 1;

for( var i = num ; i >= 1; i-- ) {
  factorial *= i ;
}
print(factorial);
```

จากตัวอย่าง หมายความว่า คำสั่งวนซ้ำจะเริ่มทำงานที่ i = num ไปจนกว่า (i >= 1) จะเป็นเท็จ โดยที่ จำนวณรอบจะลดลงทีละ 1 (i--)

2. แบบที่ใช้ in

```
for (variable_name in object){
   statement or block to execute
}
```

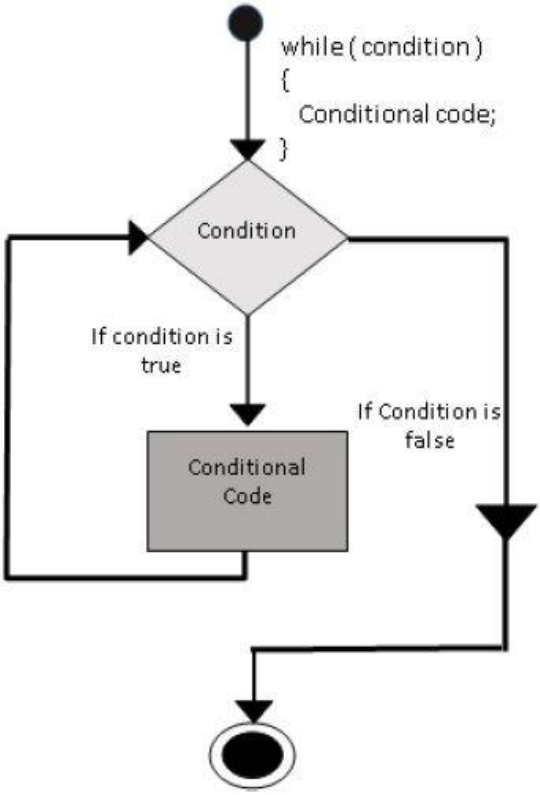
variable_name แทนถึง สมาชิกทุกตัวที่อยู่ใน object นั้น

ตัวอย่าง

```
var obj = [12,13,14];

for (var prop in obj) {
   print(prop);
}                                    // output is 12 13 14
```

รูปแบบ while



https://www.tutorialspoint.com/dart_programming/dart_programming_while_loop.htm

รูปแบบการทำงาน จะดูเงื่อนไขก่อนแล้วค่อยรันคำสั่ง
ใช้ while เป็นตัวเช็คเงื่อนไข แล้วค่อยเริ่มคำสั่ง จะทำงานวนซ้ำไปเรื่อย ๆ จนกว่าเงื่อนไขเป็น false

```
while (expression) {
   Statement(s) to be executed if expression is true
}
```

ยกตัวอย่าง

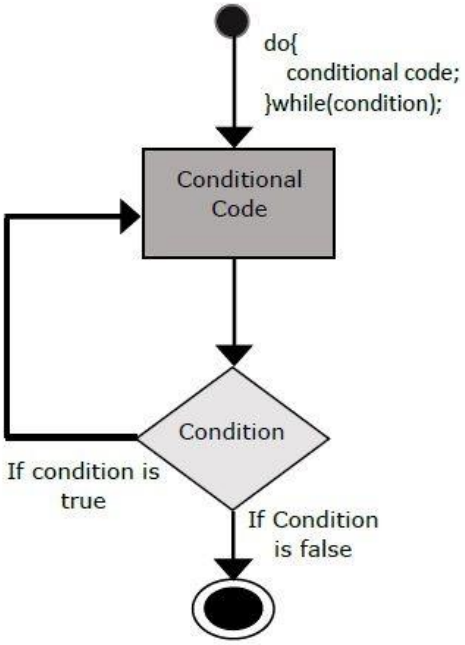
```
void main() {
   var num = 5;
   var factorial = 1;

   while(num >=1) {
      factorial = factorial * num;
      num--;
   }
   print("The factorial  is ${factorial}");
}
```

ได้คำตอบออกมาเป็น

```
The factorial is 120
```

รูปแบบ do...while



https://www.tutorialspoint.com/dart_programming/dart_programming_do_while_loop.htm

รูปแบบการทำงาน จะรันคำสั่งก่อนแล้วค่อยดูเงื่อนไข
ใช้ do เป็นตัวเริ่มคำสั่ง แล้วใช้ while เป็นตัวเช็คเงื่อนไข จะทำงานวนซ้ำไปเรื่อย ๆ จนกว่าเงื่อนไขเป็น false

```
do {
   Statement(s) to be executed;
} while (expression);
```

ยกตัวอย่าง

```
void main() {
   var n = 10;
   do {
      print(n);
      n--;
   }
   while(n>=0);
}
```

ได้คำตอบเป็น

```
10
9
8
7
6
5
```

```
4
3
2
1
0
```

Comment

คือ ข้อความที่โปรแกรมไม่สนใจ (ignored) เมื่อเวลา compile โค้ด ใช้สำหรับอธิบาย หรือบอกส่วนต่าง ๆ ของโค้ดให้ผู้พัฒนาคนอื่นเข้าใจ
โดยใช้ // แล้วตามด้วยข้อความ เมื่อต้องการ comment บรรทัดเดียว ส่วนถ้าหลายบรรทัด จะใช้ /*.... */
เช่น

```
// this is single line comment


/* This is a
   Multi-line comment
*/
```

Function

ฟังก์ชันในภาษา Dart มีความคล้ายกับ Javascript ค่อนข้างมาก เช่น

รูปแบบปกติ

```
int add (int a, int b) {
  return a + b;
}

void main() {
                print(add(3,5));
}    //คำตอบ 8
```

ตรงฟังก์ชัน add สามารถลดรูปลงเหลือ

```
add (a, b) {
  return a + b;
}
```

Arrow Function

รูปแบบ Arrow Function ก็ทำได้

```
add (a, b) => a + b;
```

Classes

การสร้าง class ใช้ keyword class มีรูปแบบดังนี้

```
class class_name {
  <fields>
  <getters/setters>
  <constructors>
  <functions>
}
```

ยกตัวอย่างการสร้างคลาส
สร้างไฟล์ class_public.dart แล้วสร้างคลาส Weapon ขึ้นมา

```
class Weapon {
 // fields
 String name= “ ”;
 int volume = 0;

 // getters/setters
 // getters
 String get weapon_name {
  return name;
 }
 int get weapon_volume {
  return volume;
 }
 // setters
 void set weapon_name(String name) {
  this.name = name;
 }
 void set weapon_volume(int volume) {
  if(volume<=10) {
   print('Go back home right now!');
  } else {
   this.volume = volume;
  }
 }
}
```

ในไฟล์ main.dart สร้าง function main() ดังตัวอย่างนี้

```
import 'class_public.dart';

void main() {
  Weapon set_1 = new Weapon();
  set_1.weapon_name = 'Rocket Flare';
  set_1.weapon_volume = 5;
  print(set_1.weapon_name);
  print(set_1.weapon_volume);
}
```

เมื่อกดรันจะได้ output เป็น

```
Go back home right now!
Rocket Flare
0
```

จะเห็นว่า มีการกำหนด *getters/setters* เพื่อกำหนดค่าให้กับตัวแปร และคืนค่าของตัวแปรที่กำหนดให้
ในส่วน main ก็มีการกำหนดค่าตัวแปรผ่าน *setter* และ statement ในนั้นจะทำงานทันที เมื่อกดรันโค้ดจะทำให้ข้อความในเงื่อนไขถูก print ออกมา ส่วน *weapon_name* กับ *weapon_volume* ก็ถูกสั่ง print ตามปกติ เพียงแต่ตัวแปร volume ไม่ได้ถูกเก็บค่าเอาไว้จาก statement ในเงื่อนไขที่อยู่ภายใน setter weapon_volume

ถ้าเปลี่ยนส่วนของ main ให้เป็นดังตัวอย่างนี้

```
void main() {
  Weapon set_1 = new Weapon();
  set_1.name = 'Rocket Flare';
  set_1.volume = 5;
  print(set_1.weapon_name);
  print(set_1.weapon_volume);
}
```

หรือตัวอย่างนี้

```
void main() {
  Weapon set_1 = new Weapon();
  set_1.name = 'Rocket Flare';
  set_1.volume = 5;
  print(set_1.name);
  print(set_1.volume);
}
```

จะได้ output เป็น

```
Rocket Flare
5
```

จากตัวอย่าง สามารถกำหนดค่าให้ตัวแปรในคลาสได้โดยตรง ทำให้ getters/setters ไม่ทำงาน และยังคืนค่าออกมาเป็นค่าที่กำหนดให้ตรง ๆ โดยไม่ผ่าน statement จึงทำให้คลาสที่สร้างมีช่องโหว่ให้คนภายนอกมากำหนดค่าได้ตามความต้องการ ซึ่งเป็นการออกแบบคลาสที่ไม่ดี จึงต้องกำหนดการทำงานใหม่ ให้สามารถกำหนดค่าได้ที่ setters เท่านั้น และได้คืนค่าผ่าน getters เท่านั้น
การกำหนดความเป็นส่วนตัว สามารถกำหนดสถานะให้กับตัวแปร จากสถานะ public ให้เป็น private นั้น สามารถกำหนด “_” ให้กับ เช่น เปลี่ยน *name (public)* เป็น *_name (private)* และ เปลี่ยน *volume (public)* เป็น *_volume (private)*

สร้างไฟล์ class_private.dart และเปลี่ยนโค้ดใหม่เป็น

```
class Weapon {
  // fields
  String _name='';
  int _volume=0;

  // getters/setters
  // getters
  String get weapon_name {
    return _name;
  }
  int get weapon_volume {
    return _volume;
  }
  // setters
  void set weapon_name(String name) {
    this._name = name;
  }
  void set weapon_volume(int volume) {
    if(volume<=10) {
      print('Go back home right now!');
    } else {
      this._volume = volume;
    }
  }
}
```

ในไฟล์ main.dart ลองกดรันดูด้วยโค้ดเดิม

```
void main() {
  Weapon set_1 = new Weapon();
  set_1.name = 'Rocket Flare';
  set_1.volume = 5;
  print(set_1.weapon_name);
  print(set_1.weapon_volume);
}
```

จะได้ output ดังนี้

```
main.dart:5:9: Error: The setter 'name' isn't defined for the class 'Weapon'.
 - 'Weapon' is from 'class_private.dart'.
Try correcting the name to the name of an existing setter, or defining a setter or field named 'name'.
  set_1.name = 'Rocket Flare';
        ^^^^
main.dart:6:9: Error: The setter 'volume' isn't defined for the class 'Weapon'.
 - 'Weapon' is from 'class_private.dart'.
Try correcting the name to the name of an existing setter, or defining a setter or field named 'volume'.
  set_1.volume = 5;
        ^^^^^^
exit status 254
```

นั่นแสดงว่า ตัวแปร name กับ volume ไม่สามารถกำหนดค่าได้จากภายนอกได้แล้ว ต้องใช้การกำหนดค่าผ่านทาง getters เท่านั้น

เปลี่ยนโค้ดที่ main.dart เป็น

```
import 'class_private.dart';


void main() {
  Weapon set_1 = new Weapon();
  set_1.weapon_name = 'Rocket Flare';
  set_1.weapon_volume = 5;
  print(set_1.weapon_name);
  print(set_1.weapon_volume);
}
```

จะได้ output เป็น

```
Go back home right now!
Rocket Flare
null
```

นี่เป็นตัวอย่างการสร้างความเป็นส่วนตัวของคลาสเพื่อความปลอดภัยของข้อมูล

นอกจากนั้น เราสามารถใช้ constructor เพื่อกำหนดค่าตัวแปร แทน setter ได้เช่นกัน

ไฟล์ class_constructor.dart

```
class Weapon {
  // fields
  String _name='';
  int _volume=0;

  // getters/setters
  // getters
  String get weapon_name {
    return _name;
  }
  int get weapon_volume {
    return _volume;
  }
  // Constructor
  Weapon(String name, int volume) {
    this._name = name;
    if(volume<=10) {
      print('Go back home right now!');
    } else {
      this._volume = volume;
    }
  }
}
```

ที่ไฟล์ main.dart ก็ลดโค้ดไปเหลือแค่นี้เอง

```
import 'class_constructor.dart';

void main() {
  Weapon set_1 = new Weapon('Rocket', 5);
  print(set_1.weapon_name);
  print(set_1.weapon_volume);
}
```

จะได้ output เป็นแบบเดิม

```
Go back home right now!
Rocket
null
```

จะเห็นได้ว่า ได้คำตอบเดียวกัน แต่ลดปริมาณโค้ดลง เนื่องจากตัด setters ออกแล้วใช้ constructor ให้กำหนดค่าแทน
แล้วเมื่อกำหนด object ใน main ก็เพียงใส่ parameter ให้ class ไปด้วยเลย ไม่ต้องเพิ่ม statement เพื่อกำหนดค่าอีก

ตัว constructor สามารถกำหนดค่าโดยที่ไม่ต้องใช้ this ได้

```
Weapon(String name, int volume) {
    _name = name;
    if(volume<=10) {
      print('Go back home right now!');
    } else {
      _volume = volume;
    }
}
```

สามารถแปลงได้อีกแบบนึง โดย constructor รับ parameter เข้ามาแล้วกำหนดไว้ในตัวแปรเลย
จากนั้น แปลง if statement นิดหน่อย จะได้คำตอบออกมาเหมือนเดิม

```
Weapon(this._name, this._volume) {
    if(_volume<=10) {
      _volume = null;
      print('Go back home right now!');
    } else {
      this._volume;
    }
}
```

ในส่วนของ class ยังมีอีกหลาย ๆ เรื่องที่จะต้องเรียนรู้ paradigm ในส่วนอื่น ๆ ของภาษา
เช่น Class Inheritance, Method Override, Static Keyword, Super Keyword

Constructor

จาก syntax ทั่วไปของ Constructor ที่มองในรูปแบบ header/body มีการแบ่งออกมาได้ดังนี้

```
//constructor head
Class_name(parameter_list) {
  //constructor body
}
```

Constructor ถูกสร้างขึ้นมาโดยไม่ต้องกำหนด data type เพื่อเป็นตัวแทนของ class ในส่วน body ใช้สร้าง statement
เพื่อกำหนดค่าให้ field เมื่อสร้าง object จากคลาส จะสามารถใส่พารามิเตอร์ให้กับ object ตั้งแต่ตอนที่สร้างขึ้นมาเลย
เช่น หากมีการสร้างคลาส Employee ที่มีการกำหนดตัวแปรสามตัวคือ ชื่อ อายุ และที่อยู่ แล้วสร้าง Constructor ที่ทำหน้าที่รับพารามิเตอร์
แต่ละค่าเอามาเก็บไว้ในตัวแปรนั้น ๆ

```
class Employee {
  // fields
  String name='';
  int age=0;
  String address='';

  // Constructor
  Employee(name, age, address) {
    this.name = name;
    this.age = age;
    this.address = address;
  }
}
```

การใช้ this เพราะต้องการให้ Constructor รู้ว่า ค่าพารามิเตอร์ที่รับเข้ามา ให้เอาไปเก็บที่ fields ของคลาส
เนื่องจากโปรแกรมจะค้นหา fields ของฟังก์ชันก่อน fields ของคลาส

ถ้าไม่มี this โปรแกรมจะเข้าใจว่าพารามิเตอร์ที่รับเข้ามาจะถูกกำหนดให้ local scope fields ของ Constructor
ซึ่งในที่นี้ถ้าไม่มีการสร้างเอาไว้ จะทำให้ field ทั้งหมดของคลาสยังคงเป็น null

เวลาจะสร้าง object จากคลาส Employee แล้วเก็บ object ไว้ในตัวแปรชื่อว่า employee_1 จะใช้ syntax แบบนี้

```
var employee_1 = new Employee("John Doe", 29, "Califonia");
```

หรือ ยกเว้น ไม่ต้องกำหนด new ก็ได้

```
var employee_1 = Employee("John Doe", 29, "Califonia");
```

คำเตือน : ต้องใส่พารามิเตอร์ให้ครบทุกค่าตามที่ฟังก์ชัน Constructor กำหนด มิเช่นนั้น จะเกิด error!!!

รูปแบบทั่วไปของการสร้าง Constructor สามารถกำหนดรูปแบบอื่น ๆ ได้อีก เช่น

Named Parameter

สามารถนำเอาแนวคิดของ Named Parameter มาผสมกับ Constructor ได้ เมื่อพารามิเตอร์มีหลายตัว

```
class Employee {
  String name;
  int age;
  String address;

  Employee({this.name=’’, this.age=0, this.address=’’});
}
```

การสร้าง object ก็ต้องใส่พารามิเตอร์ดังตัวอย่างต่อไปนี้

```
var man1 = Employee(name: 'John', age: 19, address: 'Bangkok');
```

สังเกตว่า ตอนประกาศฟังก์ชัน ต้องใส่ {} ครอบพารามิเตอร์ ส่วนตอนใช้งานฟังก์ชัน ไม่ต้องใส่ {}
สามารถใส่พารามิเตอร์สลับตำแหน่งกันได้ด้วย เช่น

```
var man1 = Employee(address: 'Bangkok', name: 'John', age: 19);
```

การใช้ Named Parameter ถือว่าเป็น Optional ซึ่งถ้าไม่ใส่ค่า จะได้ null

```
var man1 = Employee(address: 'Bangkok', age: 19);
print(man1.name);
// output is null
```

สำหรับ field ที่กำหนดเป็น private ต้องประกาศฟังก์ชัน Constructor ดังตัวอย่างต่อไปนี้

```
class Employee {
  String _name;
  int _age;
  String _address;

  Employee({String name, int age, String address}) {
    this._name = name;
    this._age = age;
    this._address = address;
  }
}
```

หรือยกเว้นไม่ต้องใส่ this แบบนี้

```
class Employee {
  String _name;
  int _age;
  String _address;

  Employee({String name, int age, String address}) {
    _name = name;
    _age = age;
    _address = address;
  }
}
```

จากนั้นก็สามารถประกาศตัวแปร object ได้เหมือนเดิม

แต่หากต้องการให้ field กำหนดเป็น final ดังตัวอย่างต่อไปนี้ ผลลัพธ์จะเป็นเช่นไร

```
class Employee {
  final String _name;
  final int _age;
  final String _address;

  Employee({String name, int age, String address}) {
    _name = name;
    _age = age;
    _address = address;
  }
}
```

สังเกตว่าจะเกิด error แน่นอน ถึงแม้ว่า จะนำ final ไปใส่ใน Constructor ก็ยัง error เหมือนเดิม ทำไมจึงเป็นเช่นนั้น ?

คำตอบคือ เนื่องจาก construction state ได้จบก่อนที่จะทำงานตรง body ดังนั้น จึงต้อง กำหนด final ให้จบตั้งแต่ header

ก่อนที่ body จะเริ่มทำงาน โดย initialize (ประกาศ) Constructor ให้เป็นดังตัวอย่างต่อไปนี้

```
class Employee {
  final String _name;
  final int _age;
  final String _address;

  Employee({String name, int age, String address}) :
    this._name = name,
    this._age = age,
    this._address = address;
}
```

สังเกตว่า หากเอา {} ที่ครอบ body ออก และเพิ่ม : เพื่อเชื่อม `_name` `_age` `_address` ให้เป็นส่วนเดียวกันกับ header และใช้ , คั่นระหว่างกลาง

ก็สามารถละ this ได้เช่นเดียวกัน

```
class Employee {
  final String _name;
  final int _age;
  final String _address;

  Employee({String name, int age, String address}) :
    _name = name,
    _age = age,
    _address = address;
}
```

ทำการสร้าง object และ การแสดงค่า

```
var man1 = Employee(name: 'John', age: 19, address: 'Bangkok');
print(man1._name);
// output is John
```

Named Constructor

Dart ได้ให้ความสามารถในการสร้าง Constructor หลาย ๆ ตัว เรียกว่า Named Constructor มีรูปแบบ syntax แบบนี้

```
Class_name.constructor_name(param_list)
```

ยกตัวอย่าง สร้างไฟล์ class_tourist.dart

```
class Tourist {
  String name;
  String address;

  //Default Constructor
  Tourist(name) {
    this.name = name;
    }

  //Named Constructor
  Tourist.touristAddress(name, address) {
    this.name = name;
    this.address = address;
    print('I am ${this.name} and I come from ${this.address}');
  }
}
```

เมื่อถึงเวลาสร้าง object 2 ตัว จากคลาส Tourist

```
import ‘class_tourist.dart’
void main() {
  var tourist = Tourist('Bonnie');
  print(tourist.name);
  var touristPresentation = Tourist.touristAddress('Puppey', 'Estonia');
}
/* output
Bonnie
I am Puppey and I come from Estonia
*/
```

เราสามารถสร้าง object โดยใช้ Constructor ตัวไหนก็ได้ จากตัวอย่าง object tourist ใช้ default constructor ส่วน touristPresentation ใช้ named constructor ซึ่งเวลาใช้ต้องเรียกออกมาตาม syntax คือ Tourist.touristAddress('Puppey', 'Estonia'); นั่นเอง

Class Inheritance

ต่อมา เราจะพูดถึงการสืบทอดคลาส (Inheritance) ซึ่งเป็นคุณสมบัติหนึ่งของคลาส โดยสืบทอดคลาส จากคลาสแม่ (Parrent or Super Class) ไปยัง คลาสลูก (Child or Sub Class) ทำให้ คลาสลูกจะได้ คุณสมบัติ (Fields) และเมธอดทั้งหมดจากคลาสแม่มาด้วย

การ inherit จะใช้ extends เป็น keyword ซึ่งมี syntax แบบนี้

```
class child_class_name extends parent_class_name
```

ยกตัวอย่าง

```
class Alcohol {
  Alcohol() {
    print('Cheers!!');
  }
}

class Beer extends Alcohol {}

void main() {
  var pileOfBeer = Alcohol();
  var pileOfBeer2 = Beer();
}

/*
Cheers!!
*/
```

การสืบทอดคลาส ปกติในภาษาโปรแกรมแบบ Imperative ประเภท OOP จะมีการสืบทอดคลาสใน 3 ลักษณะ ได้แก่
-Single คือ สืบทอดมาจากคลาสแม่เดียว
-Multiple คือ สืบทอดมาจากคลาสแม่หลายคลาส
-Multi-level คือ สืบทอดมาจากคลาสลูกอันอื่น

Dart ไม่สนับสนุนแบบ Multiple

Class Inheritance and Method Overriding

การใช้คุณสมบัติหนึ่งจากการสืบทอดคลาสคือ การเอา Method ของคลาสแม่มาดัดแปลงเป็นของตัวเอง เรียกว่า Method Overriding สามารถใช้ได้ผ่าน keyword @override

ยกตัวอย่างเช่น

```
void main() {
   Child c = new Child();
   c.m1(12);
}
class Parent {
   void m1(int a) {
     print("value of a ${a}");
   }
}
class Child extends Parent {
   @override
   void m1(int b) {
      print("value of b ${b*2}");
   }
}

/*
value of b 24
*/
```

จากตัวอย่าง เราสร้างคลาสลูกสืบทอดจากคลาสแม่ ในคลาสแม่มี method m1 เมื่อเอามาเป็น method ของลูกแล้ว สามารถดัดแปลงได้ ใส่ @override ข้างบน ก่อนทำการแปลง
ข้อจำกัดคือ
1.data type ของ method จะต้องเหมือนกัน ทั้งของแม่ของลูก
2.จำนวนพารามิเตอร์ใน method จะต้องเหมือนกัน ทั้งของแม่ของลูก
3.data type ของพารามิเตอร์ใน method จะต้องเหมือนกัน ทั้งของแม่ของลูก

ในตัวอย่าง m1 ของแม่ เป็น void มีพารามิเตอร์ 1 ตัว เป็น int ซึ่งของลูกก็ต้องเป็น void และรับค่าพารามิเตอร์ 1 ตัว เป็น int เช่นเดียวกัน

ยกตัวอย่างกรณีที่ data type ของพารามิเตอร์ใน method แม่ลูกต่างกัน

```
void main() {
  Child c = new Child();
  c.m1(12);
}
class Parent {
  void m1(int a){ print("value of a ${a}");}
}
class Child extends Parent {
  @override
  void m1(String b) {
    print("value of b ${b}");
  }
}
/*
Error: The parameter 'b' of the method 'Child.m1' has type 'String', which does not match the corresponding type, 'int', in the overridden method, 'Parent.m1'.
Change to a supertype of 'int', or, for a covariant parameter, a subtype.
*/
```

ถ้าเป็นกรณีจำนวนพารามิเตอร์ไม่เท่ากัน จะ throw error ออกมา

```
void main() {
  Child c = new Child();
  c.m1(12);
}
class Parent {
  void m1(int a){ print("value of a ${a}");}
}
class Child extends Parent {
  @override
  void m1(String b, int d) {
    print("value of b ${b} and ${d}");
  }
}
/*
Unhandled exception:
Class 'Child' has no instance method 'm1' with matching arguments.
NoSuchMethodError: incorrect number of arguments passed to method named 'm1'
*/
```